

Scalability & Future Plan

Date	30 September 2026
Team ID	SWTID-2026-5283
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Client scripts and UI Policies run in the browser on the Incident form; heavy or poorly optimised scripts (many onChange handlers, synchronous GlideAjax/server calls) slow form load.	Slower form response for agents; poor user experience when incident volume and form complexity grow.	High
2	Rules are hard-coded for the Incident table only (e.g. field visibility, mandatory and read-only conditions), so the logic is not reusable on other tables.	Extending the solution to Problem, Change or custom tables needs duplicate scripts and extra maintenance effort.	Medium
3	Limited automated testing and validation: behaviour is verified manually on a single instance, with no role-based or multi-browser test coverage.	Risk of regressions or conflicts between Client Scripts and UI Policies when new rules are added or the platform is upgraded.	Medium

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Handles a small team of fulfillers and callers working on Incident forms in a single ServiceNow instance.	Move logic to declarative UI Policies wherever possible, keep client scripts lightweight, and use scoped applications and update sets so the solution rolls out cleanly to more users, groups and instances.
2	Data Storage	Incident records and related configuration (scripts, policies, choice lists) are stored in the instance's standard tables.	Adopt archiving and data-retention rules for closed incidents, use indexed fields and reference qualifiers, and store configurable values in system properties or a custom table instead of inside scripts.
3	Performance	Client scripts and UI Policies execute on form load and on field change, with acceptable response for current usage.	Minimise onChange logic, use asynchronous GlideAjax and cached lookups, order and consolidate UI Policies, and monitor form load time with browser developer tools and Performance Analytics.
4	Security	Access is controlled through standard roles and ACLs; validation is mainly done on the client side.	Enforce the same rules server side with Data Policies, Business Rules and ACLs (client checks can be bypassed), apply role-based UI Policy conditions, and review scripts before promotion to production.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Current release: Client Script and UI Policy on the Incident form (dynamic fields, mandatory and read-only rules, validation messages).	Completed (Sep 2026)	Guided, consistent incident data entry with fewer errors.
Phase 2	Optimise scripts, add server-side validation (Data Policy / Business Rules) and refactor rules into reusable script includes.	Oct – Nov 2026	Faster forms and stronger, tamper-proof data quality.
Phase 3	Extend the solution to Problem and Change tables, with role-based rules and a mobile / Service Portal friendly experience.	Dec 2026 – Feb 2027	Wider adoption with a consistent experience across ITSM modules.
Phase 4	Add automated testing (ATF), a dashboard for form performance and rule usage, and AI-assisted suggestions for incident fields.	Mar – Jun 2027	Lower maintenance effort, safer upgrades and smarter incident handling.